

Technical Analysis Report

Context Management and Persistent Memory Architecture for ConAI

Papers analyzed

- 1. ACON — Agent Context Optimization
- 2. Active Context Compression — Focus Agent
- 3. HyMem — Hierarchical Context Management via Information Isolation
- 4. LCM — Lossless Context Management
- 5. MEMORI — Persistent Memory Layer for Context-Aware LLM Agents

1. Executive conclusion

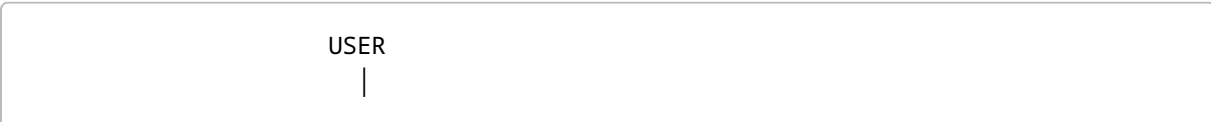
These five papers are **not five competing implementations of the same thing**. They solve different layers of the long-running-agent memory problem.

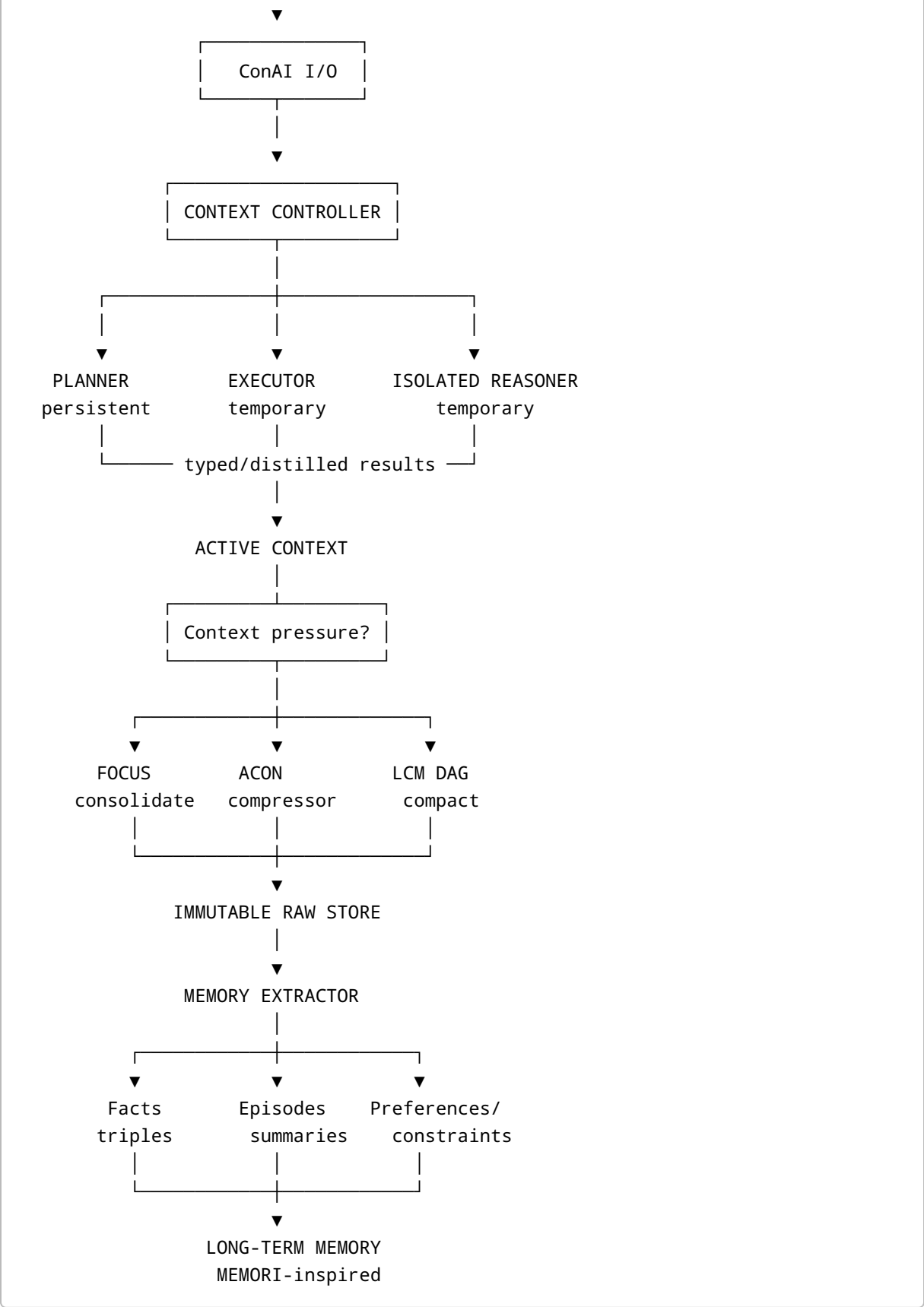
The most useful interpretation for ConAI is:

Paper	Main problem solved	Best concept to take
ACON	What information should compression preserve?	Failure-driven, optimized compression policy
Focus	When should an agent voluntarily discard working history?	Explicit focus phases + consolidation
HyMem	How do we prevent context pollution before it occurs?	Planner/executor/reasoner isolation
LCM	How do we compress without permanently losing information?	Immutable history + hierarchical summary DAG
MEMORI	How do we remember useful information across sessions?	Structured semantic long-term memory

The strongest ConAI architecture should therefore **not select one paper**.

It should combine them:





The central design principle is:

Raw history is ground truth. Active context is a temporary computational workspace. Summaries are views over history. Long-term memory is structured knowledge extracted from history.

That distinction is extremely important.

2. The actual problem these papers solve

A conventional agent effectively does:

```
history = []  
  
while running:  
    history.append(user_input)  
  
    response = llm(  
        system_prompt +  
        history  
    )  
  
    history.append(response)
```

After hundreds of interactions:

```
System prompt  
User message 1  
Assistant response 1  
Tool output 1  
User message 2  
...  
Tool output 91  
Failed attempt  
Correction  
Search result  
Search result  
...  
Current user message
```

Three things happen.

Computational problem

Every generation repeatedly processes old tokens.

Long histories therefore increase prefill cost, KV-cache requirements and latency.

Reasoning problem

Important information becomes a progressively smaller proportion of the prompt.

HyMem formalizes this as **context dilution**:

$$\rho_t = \frac{|Z_t|}{|C_t|}$$

where Z_t is planning-relevant information and C_t is total context.

Memory problem

Simply deleting old context solves token growth but creates forgetting.

This creates the fundamental design problem:

```
KEEP EVERYTHING
  ↓
accurate history
but
huge context

DELETE HISTORY
  ↓
small context
but
information loss
```

The five papers attack different parts of this trade-off.

3. ACON — Agent Context Optimization

3.1 Core idea

ACON's central insight is subtle:

Compression itself is not the hard problem. Learning what the compressor must preserve is the hard problem.

A generic instruction such as:

```
Summarize the conversation concisely.
```

is dangerous for an agent.

An ordinary summarizer might discard:

```
/tmp/build_319/config.json
```

because it looks insignificant.

But if the next tool call requires that exact path, losing it destroys the trajectory.

Agent context can contain:

- state variables
- file paths
- tool parameters
- causal relationships
- failed actions
- successful actions
- prerequisites
- account states
- deadlines
- intermediate results
- decisions
- unresolved problems

ACON explicitly argues that this heterogeneous state makes ordinary summarization insufficient.

3.2 Two independent compression mechanisms

ACON compresses **history** and **observations** separately.

History compression

Let:

$$h_t = (o_0, a_0, o_1, a_1, \dots)$$

When history exceeds threshold T_{hist} :

$$h'_t = \begin{cases} f(h_t; \phi, P_{hist}) & |h_t| > T_{hist} \\ h_t & otherwise \end{cases}$$

The compressed history replaces the previous raw history in the model's active context.

Recommended experimental threshold:

History $\approx$ 4096 tokens

ACON's ablation found 4096 history tokens and 1024 observation tokens to be good trade-offs in its tested environments.

These should **not** be blindly hard-coded into ConAI; they are benchmark-derived values.

3.3 Observation compression

Tool observations can themselves be enormous.

Example:

```
ConAI
  ↓
search()
  ↓
15,000 token result
```

Instead of placing all 15k tokens into history:

$$o'_t = f(o_t, h_{t-1}; \phi, P_{obs})$$

when:

$$|o_t| > T_{obs}$$

Notice the important detail:

Observation compression receives previous history too.

That means the compressor can determine relevance relative to the current task rather than compressing the observation blindly.

3.4 ACON's optimization objective

The authors formulate compression as:

$$\max_{\psi} E[R(s_T(\psi))] - \lambda E[C(H'(\psi))]$$

Conceptually:

```
maximize task success
-
λ × context cost
```

The compressor should therefore not minimize token count alone.

An extremely small summary that destroys task success is a **bad compressor**.

This should become a ConAI design invariant.

3.5 Failure-driven compression optimization

This is ACON's most valuable idea.

Take a training task.

Run:

```
RUN A
No compression
→ SUCCESS
```

Then:

```
RUN B
Compression
→ FAILURE
```

Now the system knows the failure is plausibly associated with information removed or distorted by compression.

The optimizer compares:

Original history H

vs

Compressed history H'

and asks an LLM to diagnose:

What information disappeared?

What state was corrupted?

Which variable was lost?

Which causal relation disappeared?

Which facts were summarized incorrectly?

The paper explicitly constructs a contrastive subset from cases where the uncompressed agent succeeds and compressed agent fails.

3.6 Natural-language gradient

Instead of updating neural-network weights, ACON generates textual feedback.

Conceptually:

Failure:

The compressor removed the generated filename
required by the next tool call.

Then the compression guideline changes from:

Keep important information.

to something more specific:

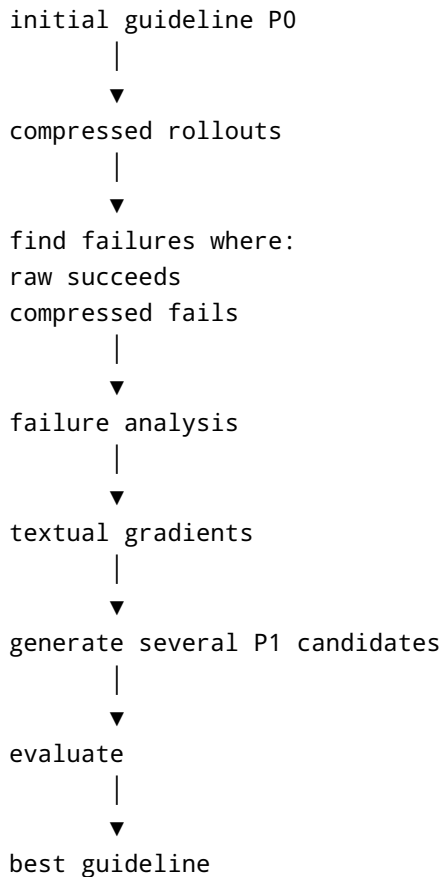
Preserve all runtime identifiers, file paths, generated IDs, variable values and parameters that may be required by subsequent actions.

The paper describes this feedback as a **natural-language gradient**. Multiple failure reports are aggregated to generate new candidate compression guidelines.

3.7 UT — Utility Maximization

The first optimization phase is **UT**.

Pipeline:



UT primarily optimizes **task success**.

3.8 CO — Compression Maximization

Once compression works reliably, ACON asks another question:

What information did we retain that the successful agent never actually needed?

Only successful compressed trajectories are considered.

This produces another refinement:

```
P1
↓
analyze unnecessary retained information
↓
P2
```

The CO step therefore pushes toward shorter context after UT has established utility.

A useful mental model is:

```
UT:
"Don't delete anything necessary."

CO:
"Now remove everything unnecessary."
```

3.9 Important ACON finding for ConAI

ACON found that simultaneously maximizing both history and observation compression can reduce tokens further but can also substantially degrade performance.

Therefore ConAI should **not blindly compress every channel**.

A context policy should determine whether:

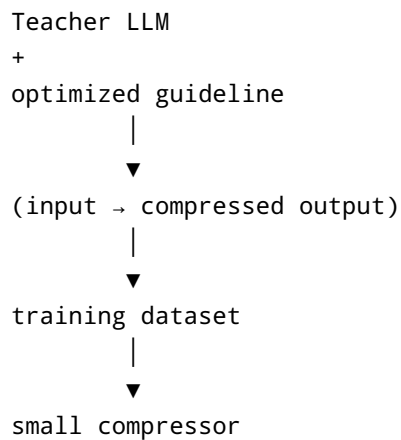
```
history compression
observation compression
both
neither
```

is appropriate.

3.10 Compressor distillation

Running a large model purely to summarize context wastes resources.

ACON therefore uses the optimized large-model compressor as a teacher.



The student minimizes normal token-level cross entropy on successful teacher compression examples.

They distilled to models including:

- Qwen3-14B
- Qwen3-8B
- Phi-4

and report >95% retention of teacher performance.

The reported LoRA setup is also unusually useful for reproduction:

```
LoRA rank = 16
alpha = 32
learning rate = 1e-4
epochs = 3
batch = 4
max sequence = 10,000
warmup = 5%
weight decay = 0.01
```

```
optimizer = AdamW
decoding temperature = 0
```

3.11 What ConAI should take from ACON

Do **not** initially reproduce the full training pipeline.

Implement:

```
class CompressionPolicy:
    history_threshold
    observation_threshold
    guideline_version

class Compressor:
    compress_history()
    compress_observation()

class CompressionEvaluator:
    compare_successful_and_failed_runs()
    identify_information_loss()
```

Start with hand-written structured guidelines.

After you have enough ConAI benchmark trajectories, implement ACON-style automatic guideline optimization.

4. Active Context Compression — Focus

Focus attacks the problem differently.

Instead of:

System decides when context is too large.

Focus says:

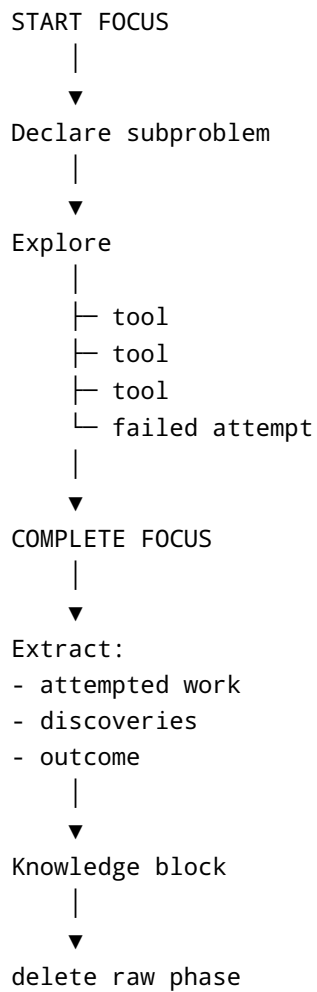
Agent recognizes that a reasoning phase is complete and voluntarily consolidates it.

4.1 Focus lifecycle

Two operations are added:

```
start_focus()
complete_focus()
```

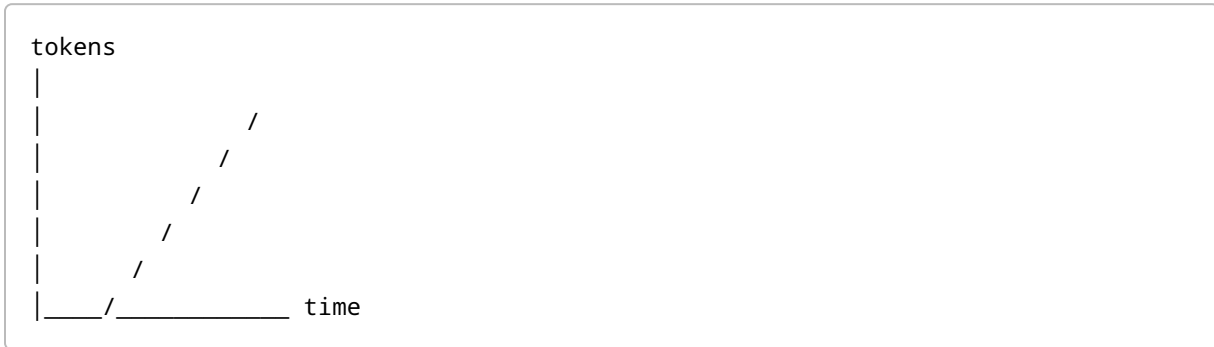
The loop is:



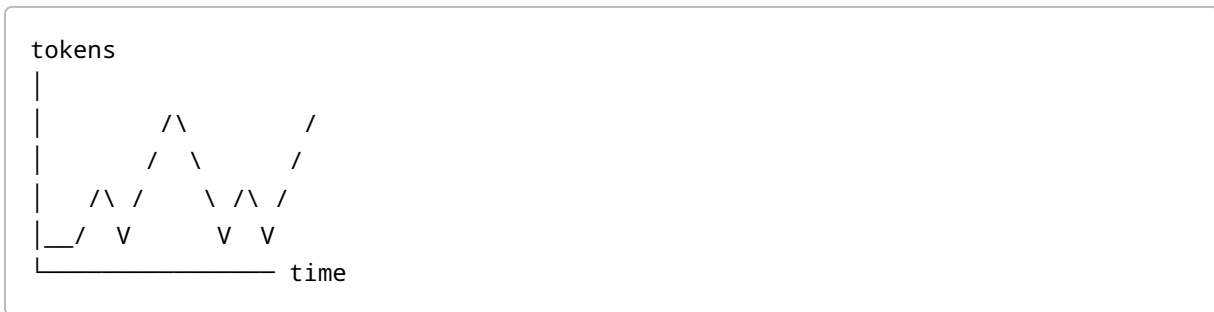
The paper describes exactly this checkpoint → exploration → consolidation → withdrawal sequence.

4.2 Sawtooth context

Normal agent:



Focus:



Context repeatedly grows and collapses.

Hence **sawtooth context growth**.

4.3 Knowledge block

After a phase, the system keeps something like:

FOCUS: Investigate database error

Attempted:

- inspected connection module
- tested credentials
- inspected environment variables

Learned:

- DB server is reachable
- credentials valid

- application uses stale DB_HOST

Outcome:

- update DB_HOST before continuing

The raw 40–80 messages can then disappear from active context.

4.4 Critical contradiction inside the paper

The conceptual description says the model has autonomy and no timer/heuristic forces compression.

However, the successful experimental configuration **did use aggressive external scaffolding**:

```
always start_focus before exploration
complete_focus after 10-15 tool calls
system reminder after 15 calls
4-6 structured phases
```

That distinction matters.

The paper demonstrates **agent-controlled compression with policy scaffolding**, not purely spontaneous self-management.

4.5 Experimental result

Focus obtained:

```
Baseline tokens = 14.92M
Focus tokens    = 11.53M

reduction       = 22.7%

accuracy:
baseline = 60%
Focus    = 60%
```

with about six compressions and 70.2 dropped messages per task.

4.6 Major weakness

One task produced:

+110% token usage

because compression removed useful context and caused re-exploration.

This gives us an important architectural rule:

Compression frequency cannot be based solely on elapsed turns.

The system should also estimate **state continuity**.

For example:

```
exploration phase
→ aggressive compression allowed

iterative debugging
→ conservative compression

final verification
→ preserve recent evidence
```

4.7 ConAI implementation

Add:

```
class FocusManager:

    start_focus(goal)

    add_event(event)

    should_complete_focus()

    complete_focus()

    consolidate()
```

```
archive_raw_phase()
```

Possible state:

```
FocusSession(  
    id,  
    objective,  
    start_message_id,  
    tool_calls,  
    status,  
    summary,  
    artifacts,  
)
```

But unlike the original Focus paper:

do not permanently delete the raw phase.

Archive it into LCM-style immutable storage.

That immediately eliminates one of Focus's largest risks.

5. HyMem — Hierarchical Context Management

For ConAI, **HyMem is probably the most important architectural paper of these five.**

Its insight is:

Don't wait for context to become polluted and then summarize it. Prevent noisy information from entering the planner in the first place.

5.1 Flat ReAct is architecturally wrong for long tasks

Normal:

```
Planner reasoning  
Tool request
```

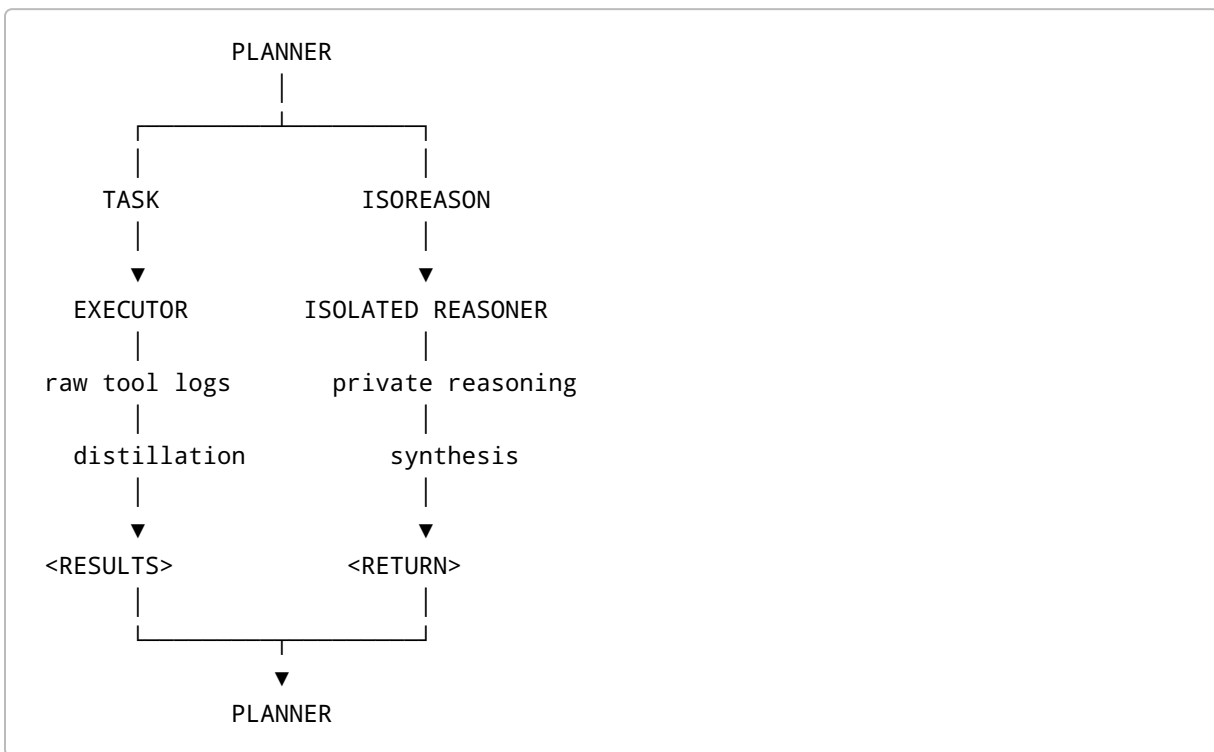
Massive search result
Planner reasoning
Tool error
Retry
Massive result
Reasoning
...

Everything occupies one context.

HyMem separates these channels.

5.2 HyMem architecture

The page-4 architecture defines:



Only the planner persists. Executor and isolated-reasoning contexts are created on demand and destroyed after returning typed results.

This is a major difference from summarization-only architectures.

5.3 Planner context

Planner contains:

$$C_t^{(p)} = (P^{(p)}, q, M^{epi}, M^{wm}, M^{tool}, H_t^{(p)})$$

So persistent planning context consists of:

```
system/planner prompt
user query
episode memory
working memory
tool memory
typed results from previous subtasks
```

It does **not** contain raw browser/search/tool noise.

5.4 Planner actions

HyMem restricts the planner to four conceptual operations:

```
<TASK>
<ISOREASON>
<FOLD>
<ANSWER>
```

This is excellent architecture.

TASK

Delegate environmental/tool work.

ISOREASON

Create an isolated deep reasoning environment.

FOLD

Consolidate memory.

ANSWER

Finish.

5.5 Executor context

For a task d :

$$C_{t,0}^{(e)} = (P^{(e)}, d, M^{tool})$$

Notice what is missing:

```
entire planner history
entire conversation
episode memory
raw previous traces
```

The executor gets only what it needs.

It may perform several tool calls.

Then raw observations are filtered.

The paper explicitly preserves:

```
named entities
dates
numbers
source identifiers
evidence
answer candidates
```

while removing:

```
navigation
duplicates
formatting noise
irrelevant snippets
tool-error boilerplate
```

5.6 Typed executor return

Instead of returning raw logs:

```
<RESULTS>
findings:
evidence:
sources:
status:
gaps:
</RESULTS>
```

Only this object enters planner history.

This single idea can dramatically clean ConAI context.

5.7 Isolated reasoning

Some problems require substantial internal analysis.

Instead of contaminating planner context:

```
Planner
|
▼
spawn isolated reasoning session
|
├─ reason
├─ retrieve
├─ test hypotheses
├─ reconcile evidence
└─ reason
|
▼
<RETURN>
conclusion
supporting facts
sources
confidence
assumptions
</RETURN>
```

Then destroy the temporary context.

The formal return includes confidence κ , supporting facts, sources and assumptions.

This architecture is highly suitable for ConAI's research mode.

5.8 Three memory types

HyMem uses:

Episode memory

```
completed milestones  
verified facts  
important decisions
```

Working memory

```
current objective  
unresolved gaps  
next plan
```

Tool memory

```
successful tool patterns  
failed queries  
tool-specific caveats
```

This is far better than one generic:

```
memory = []
```

5.9 Asymmetric memory injection

Not every subsystem receives every memory.

Planner:

```
episode + working + tool
```

Isolated reasoner:

```
episode + working + tool
```

Executor:

```
tool only
```

That is a crucial principle:

Memory retrieval should be role-dependent.

5.10 Fold trigger

HyMem gives one of the most directly implementable policies in all five papers.

Memory folding occurs if the planner requests it OR:

$$r_{ctx} > 0.99$$

or:

```
executor calls >= 5  
consecutive failures >= 3  
planner turns >= 8  
tool calls >= 10
```

Again, these values are paper settings, not universal constants.

For ConAI, make them configuration parameters.

5.11 HyMem result

With DeepSeek-V4:

GAIA
ReAct: 56.6
HyMem: 66.7

BrowseComp+
ReAct: 56.6
HyMem: 61.3

Hard BrowseComp+:

14% → 30%

Interestingly, Qwen3-32B did not beat ReAct on BrowseComp+, showing that architectural sophistication still depends on the base model's ability to exploit the structure.

6. LCM — Lossless Context Management

LCM solves a different and extremely important problem:

What happens when a summary is wrong?

Most compression systems say:

```
raw messages
  ↓
summary
  ↓
delete raw messages
```

LCM says that is unacceptable.

6.1 Dual-state architecture

LCM maintains:

IMMUTABLE STORE

and

ACTIVE CONTEXT

The immutable store contains every original:

```
user message
assistant message
tool result
```

verbatim.

The active context contains:

```
recent raw messages
+
older summaries
```

Therefore:

Compression ≠ deletion

This is arguably the single most important reliability rule for ConAI.

6.2 Summary DAG

Instead of one rolling summary:

```
Messages 1-20
  |
  ▼
Summary A

Messages 21-40
  |
  ▼
Summary B

Messages 41-60
  |
```

▼
Summary C

Later:

```
Summary A ┌  
Summary B ┤─> Condensed Summary X  
Summary C └
```

And eventually:

```
X + Y + Z  
  |  
  ▼  
higher-level summary
```

This forms a hierarchical **Directed Acyclic Graph**.

The original data remains reachable beneath every summary.

The database model separates full-fidelity messages from leaf and condensed summaries and maintains provenance pointers.

6.3 Why this is superior to simple RAG

Flat vector RAG can retrieve a semantically similar chunk but lose:

```
who said it  
what happened before it  
what decision followed it  
which later event invalidated it
```

LCM's hierarchy gives:

```
high-level history map  
  ↓  
summary  
  ↓  
sub-summary
```

↓
original messages

So retrieval becomes **multi-resolution navigation**.

6.4 Soft and hard thresholds

LCM uses two thresholds.

```
context < soft threshold
→ nothing

soft <= context < hard
→ asynchronous compression

context >= hard
→ blocking compression
```

This is much better than a single threshold.

For ConAI:

```
if usage < 0.60:
    PASS

elif usage < 0.80:
    schedule_async_compaction()

else:
    compact_before_next_generation()
```

Those percentages are a proposed ConAI policy, **not LCM's reported values**.

6.5 Three-level summarization escalation

LLMs sometimes produce:

```
10,000 input tokens  
→  
11,000-token "summary"
```

LCM explicitly guards against this.

Level 1

Normal detail-preserving summary.

Level 2

Aggressive bullet summary targeting roughly half the previous target.

Level 3

Deterministic truncation to 512 tokens.

Therefore compression is guaranteed eventually to shrink data.

Excellent production engineering.

6.6 Retrieval interfaces

LCM exposes:

```
lcm_grep()
```

Regex search over immutable history.

```
lcm_describe()
```

Inspect metadata for summary/file IDs.

```
lcm_expand()
```

Recover original content represented by a summary.

Importantly, expansion is restricted to sub-agents so that a main agent cannot accidentally flood its own context with enormous old history.

This should absolutely influence ConAI.

6.7 Large-file handling

Files over a threshold—25k tokens by default in the paper—are not loaded directly into active context.

Instead:

```
large file
├── external filesystem
└── active context:
    ├── file ID
    ├── path
    └── exploration summary
```

Different file types receive different exploration strategies:

```
JSON → schema/shape
CSV → schema/shape
SQL → structural inspection
Code → signatures/classes
Text → semantic summary
```

This maps extremely well onto ConAI's research-document architecture.

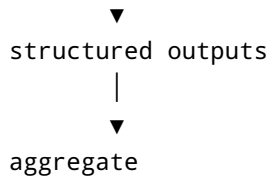
6.8 Recursive task partitioning

LCM isn't only memory compression.

It also prevents giant datasets from entering the agent context.

`llm_map`:

```
dataset
├── item 1 → LLM
├── item 2 → LLM
├── item 3 → LLM
└── ...
    │
```



The engine manages:

```
parallelism
schema validation
retries
storage
```

Default worker concurrency in the described implementation is 16.

6.9 `agentic_map`

For complex items:

```
item
↓
temporary sub-agent
↓
tools + reasoning
↓
structured answer
```

The parent never sees intermediate noise.

Again, this converges conceptually with HyMem.

6.10 Infinite recursion prevention

LCM introduces a very clever invariant.

A subagent delegating again must specify:

```
delegated_scope
kept_work
```

If it delegates its entire responsibility, the engine rejects it.

Therefore:

```
parent task
  ↓
smaller task
  ↓
smaller task
  ↓
eventually direct execution
```

The paper argues this gives structural termination without an arbitrary recursion-depth limit.

7. MEMORI — Persistent Long-Term Memory

The first four papers mostly address **intra-task context**.

MEMORI addresses:

```
Session 1
Session 2
Session 3
...
Session 500
```

That is a different problem.

7.1 Core insight

MEMORI argues:

Memory is primarily a **data structuring problem**, not merely a storage problem.

Raw conversational RAG:

```
conversation
↓
chunk
↓
embedding
↓
vector DB
```

creates a noisy embedding space containing:

```
pleasantries
corrections
repetitions
irrelevant tangents
contradictory statements
```

MEMORI transforms conversation before indexing it.

7.2 Advanced Augmentation

Two representations are generated.

Semantic triples

Example:

```
"Swastik prefers Qwen3 for local inference."
```

becomes conceptually:

```
subject    = Swastik
predicate  = prefers_model
object     = Qwen3
```

A second statement:

```
"ConAI uses Ollama."
```

becomes:

```
ConAI
uses_runtime
Ollama
```

MEMORI extracts facts, preferences, constraints and evolving attributes into subject-predicate-object triples and links them back to their source conversation.

7.3 Why triples alone are insufficient

Suppose memory contains:

```
Swastik → prefers → Qwen
```

It does not explain:

```
why?
for which project?
under what hardware constraints?
was this later changed?
```

Therefore MEMORI simultaneously creates **conversation summaries**.

Triples provide:

```
precision
```

Summaries provide:

```
narrative
temporal context
intent
decision evolution
```

The paper explicitly describes this as a dual-layer memory representation.

7.4 Retrieval

In the benchmark implementation, triples were:

```
embedded
+
indexed in FAISS
```

using Gemma-300 embeddings.

Retrieval combines:

```
cosine similarity
+
BM25
```

That hybrid search matters.

Vector-only retrieval is weak for exact values such as:

```
Qwen3-4B
4096
config_manager.py
```

BM25 helps preserve lexical exactness.

7.5 Temporal reasoning

The MEMORI answer prompt explicitly instructs the model to:

```
consider timestamps
prefer recent contradictory memories
resolve relative dates
use triples for exact facts
use summaries for broader context
```

This is especially important for a personal assistant because preferences and project states evolve.

Memory must therefore support:

```
created_at
valid_from
valid_to
supersedes
source
confidence
```

The latter fields are my recommended extension; the paper's prompt directly demonstrates the need for timestamp-aware contradiction handling.

7.6 Performance

MEMORI reports:

Single-hop	87.87%
Multi-hop	72.70%
Open-domain	63.54%
Temporal	80.37%
Overall	81.95%

versus a full-context ceiling of 87.52%.

More importantly:

Full context	= 26,031 tokens
MEMORI	= 1,294 tokens

or about:

4.97% of full context

7.7 Weaknesses

MEMORI itself shows where the structured approach struggles.

Temporal

80.37%, below LangMem's 86.92% and Zep's 83.33%.

Triples naturally represent static facts better than evolving timelines.

Open-domain

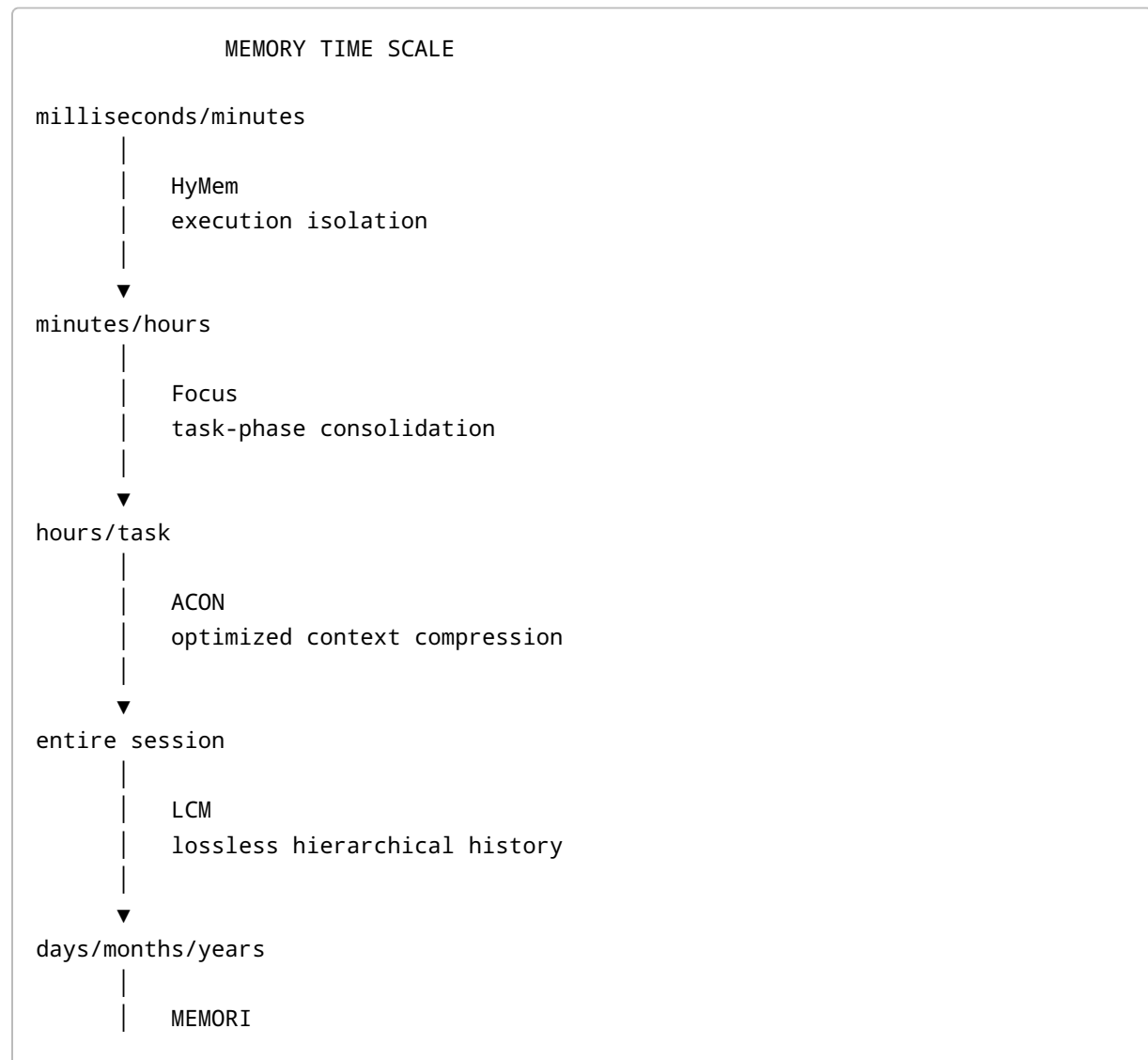
63.54%.

Broad questions lack obvious retrieval anchors and require wider synthesis.

Therefore ConAI should never rely solely on semantic triples.

8. How the five papers fit together

The cleanest abstraction is:



| persistent semantic memory
▼

They are complementary.

9. Recommended ConAI memory hierarchy

I recommend six layers.

```
L0 RAW IMMUTABLE EVENT STORE
L1 ACTIVE CONTEXT
L2 WORKING MEMORY
L3 EPISODIC MEMORY
L4 SEMANTIC MEMORY
L5 ARCHIVAL SUMMARY DAG
```

10. L0 — Immutable Event Store

Inspired primarily by LCM.

Never modify or destroy raw events.

Example database:

```
events

event_id
session_id
timestamp
role
event_type
content
token_count
tool_name
parent_event_id
metadata
```

Event types:

```
USER
ASSISTANT
TOOL_CALL
TOOL_RESULT
SYSTEM
MEMORY_EVENT
COMPRESSION_EVENT
FOCUS_START
FOCUS_END
```

This becomes ConAI's **source of truth**.

11. L1 — Active Context

Only information required for the current generation.

Example:

```
System prompt
Current goal
Working memory
Relevant semantic memories
Relevant episode memory
Planner history
Last few raw turns
Current user query
```

Target:

```
small
high-signal
dynamic
```

12. L2 — Working Memory

HyMem-inspired.

Schema:

```
WorkingMemory:
  current_goal
  current_subgoal
  constraints
  unresolved_questions
  current_plan
  temporary_variables
  required_artifacts
  next_actions
```

This memory is mutable.

It describes **what ConAI is currently doing**.

13. L3 — Episodic Memory

Store events and milestones:

```
Implemented benchmark module
Changed context threshold
Benchmark revealed 30-second cold start
Added warm-start architecture
Selected STT system
```

Each episode should contain:

```
Episode:
  id
  session_id
  title
  timestamp
  goal
  events
  decisions
  result
  unresolved_items
  source_event_ids
```

14. L4 — Semantic Memory

MEMORI-inspired.

Schema:

```
MemoryFact:
  id
  subject
  predicate
  object
  created_at
  valid_from
  valid_to
  confidence
  source_event_ids
  supersedes
  embedding
```

This supports precise persistent recall.

15. L5 — Summary DAG

LCM-inspired.

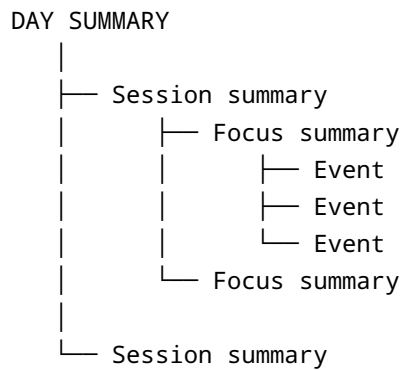
```
SummaryNode:
  id
  level
  summary
  token_count

  child_message_ids
  child_summary_ids

  start_time
  end_time

  entities
  topics
  file_ids
```

Possible hierarchy:



Nothing is truly lost.

16. Context assembly engine

Every inference should go through a **ContextAssembler**.

```

context = assembler.build(
    query=user_input,
    session=session,
    token_budget=budget
)
  
```

It should allocate budget approximately as:

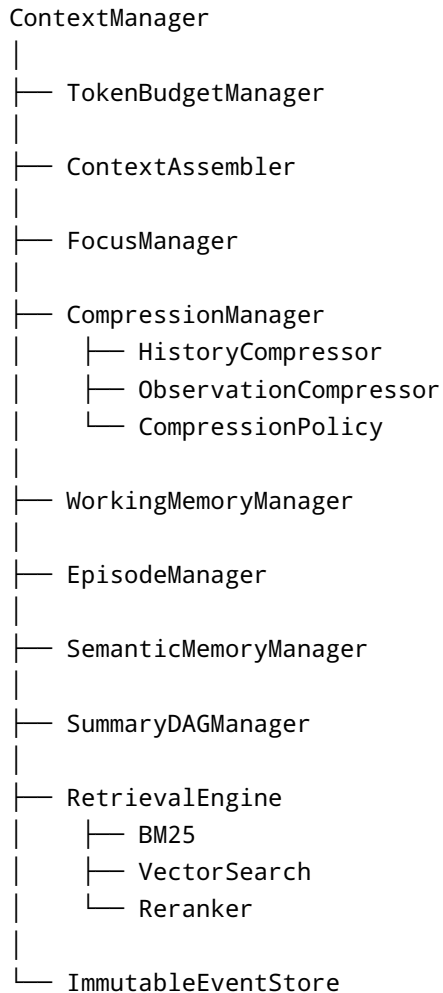
```

SYSTEM
↓
CURRENT GOAL
↓
WORKING MEMORY
↓
RECENT RAW CONTEXT
↓
RETRIEVED SEMANTIC MEMORY
↓
RETRIEVED EPISODES
↓
HISTORICAL SUMMARIES
↓
USER QUERY
  
```

Do **not** simply concatenate everything returned by retrieval.

17. Proposed Context Manager architecture

Your current `ContextManager` should eventually become an orchestrator rather than a simple message-trimming class.



18. Proposed runtime pipeline

User sends request

```
User
↓
Session Manager
↓
store raw event
```

Memory retrieval

```
Query
↓
query classifier
↓
memory requirements
↓
hybrid retrieval
```

Retrieve:

```
working state
semantic facts
relevant episodes
historical summaries
```

Planner context

```
system
+ user query
+ working memory
+ retrieved memory
+ compact planner history
```

Planner decides

```
TASK
ISOREASON
```

ANSWER
FOLD

Tool execution

Tool output stays in temporary executor context.

```
tool output
↓
observation compressor
↓
structured evidence
```

Only evidence reaches planner.

Context pressure

```
TokenBudgetManager
|
▼
soft threshold?
|
├── no → continue
|
└── yes
    ↓
    async compression
```

Hard threshold:

```
block
↓
compact
↓
continue
```

Focus completion

When a logical phase finishes:

```

raw focus trajectory
    ↓
ACON-guided consolidation
    ↓
Focus summary
    ↓
Summary DAG

```

Memory extraction

Background process:

```

new events
    ↓
Advanced Augmentation
    ↓
facts
episodes
summaries
    ↓
memory DB

```

19. Retrieval algorithm

Do not implement:

```
top_k = vector_db.search(query)
```

alone.

Use:

$$Score = w_s S_{semantic} + w_b S_{BM25} + w_r S_{recency} + w_i S_{importance} + w_t S_{temporal}$$

Potential initial values:

```

semantic    0.35
BM25        0.25
recency     0.15

```

```
importance 0.15
temporal   0.10
```

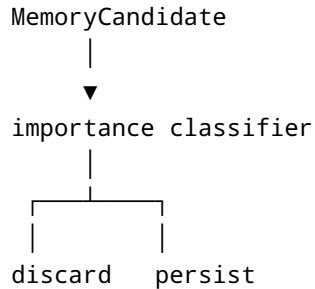
These are **proposed ConAI engineering defaults**, not values reported by the papers.

Then rerank.

20. Memory write decision

Not every statement should become permanent memory.

Introduce:



Persist information such as:

```
stable preferences
project decisions
constraints
important entities
completed milestones
reusable procedures
important failures
```

Avoid persisting:

```
greetings
temporary wording
repeated facts
irrelevant chatter
raw chain-of-thought
```

21. Contradiction handling

Suppose:

```
T1:  
ConAI uses model A.  
  
T2:  
ConAI now uses model B.
```

Do not delete A.

Represent:

```
Fact A  
valid_from = T1  
valid_to   = T2  
  
Fact B  
valid_from = T2  
valid_to   = NULL  
supersedes = Fact A
```

Then questions such as:

```
"What model do we use now?"
```

retrieve B.

While:

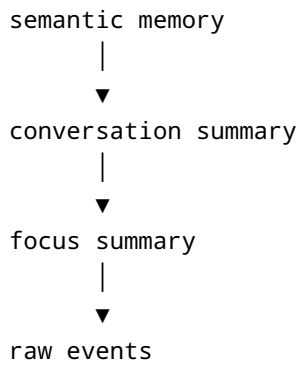
```
"What model were we using before?"
```

can retrieve A.

This improves on simple triple stores and directly addresses the temporal weakness visible in MEMORI's results.

22. Memory provenance

Every generated memory must point backward:



Therefore the model can answer:

Memory says X.

but if confidence is insufficient:

```
expand source
↓
inspect raw conversation
↓
verify X
```

This combines MEMORI and LCM exceptionally well.

23. Compression provenance

Every summary should store:

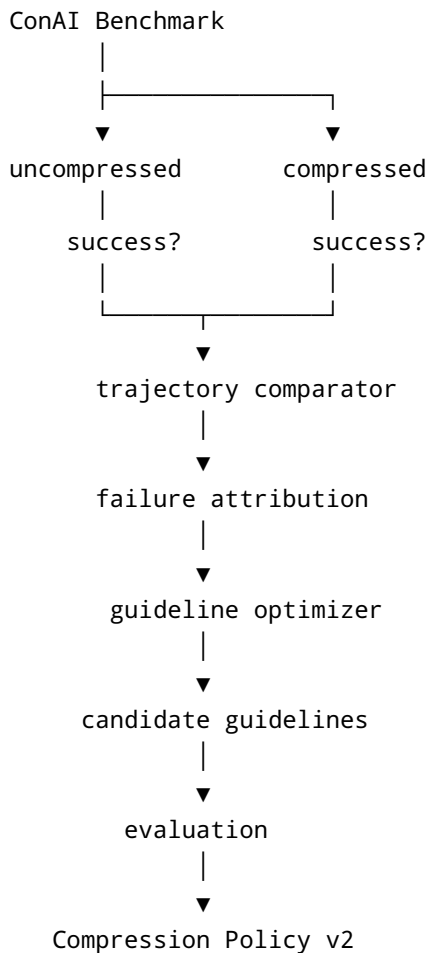
```
CompressionRecord:
  summary_id
  compressor_model
  guideline_version
  timestamp
  original_token_count
  compressed_token_count
```

```
source_ids
compression_ratio
```

This is essential for your research work because it lets you determine which compressor versions caused failures.

24. ACON-style self-improving compressor for ConAI

Once enough benchmark runs exist:



This can become a genuinely interesting ConAI research component.

25. Context controller modes

Your planned manual context controller should expose meaningful modes rather than arbitrary token numbers.

For example:

```
/context auto  
/context precise  
/context balanced  
/context aggressive  
/context raw
```

RAW

No compression.

Useful for benchmarking.

PRECISE

Conservative compression.

Good for:

```
research  
coding  
technical reasoning
```

BALANCED

Normal operation.

AGGRESSIVE

Maximum compression for:

```
casual conversation  
low-resource operation  
very long sessions
```

AUTO

Controller predicts appropriate policy from:

```
task type
context size
retrieval density
recent failures
tool activity
phase
```

26. Context pressure metric

Do not monitor tokens alone.

Create:

$$P = \alpha U + \beta R + \gamma N + \delta D$$

where:

```
U = token utilization
R = redundancy
N = accumulated execution noise
D = distraction risk
```

Then:

```
P < threshold1
→ nothing

threshold1 < P < threshold2
→ async fold

P > threshold2
→ immediate fold
```

This is a proposed extension beyond the five papers and could itself become part of ConAI's research novelty.

27. Focus + HyMem combination

Focus and HyMem should not be implemented independently.

Use:



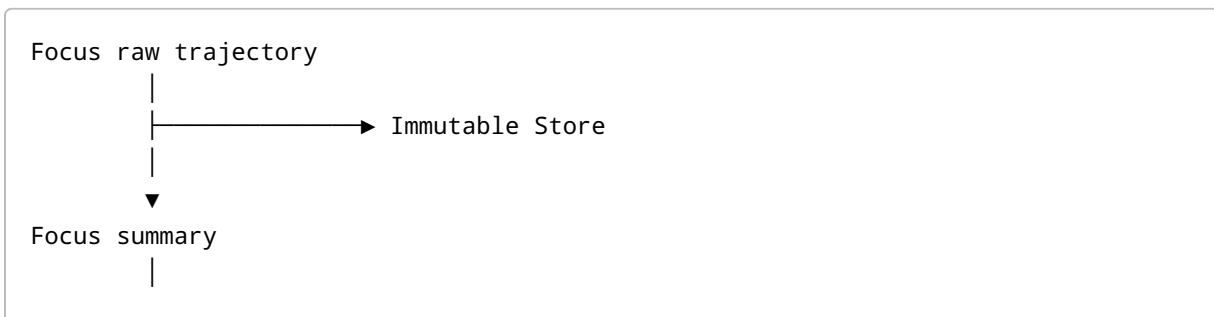
Thus Focus becomes a **semantic boundary around a HyMem execution phase**.

That is cleaner than using an arbitrary "every 10 messages" rule.

28. Focus + LCM combination

Original Focus deletes history.

ConAI should instead:



▼
Active Context

So:

active deletion
≠
physical deletion

This eliminates destructive forgetting.

29. ACON + LCM combination

ACON determines:

HOW TO SUMMARIZE

LCM determines:

HOW TO STORE THE SUMMARY
AND RECOVER ITS ORIGINAL

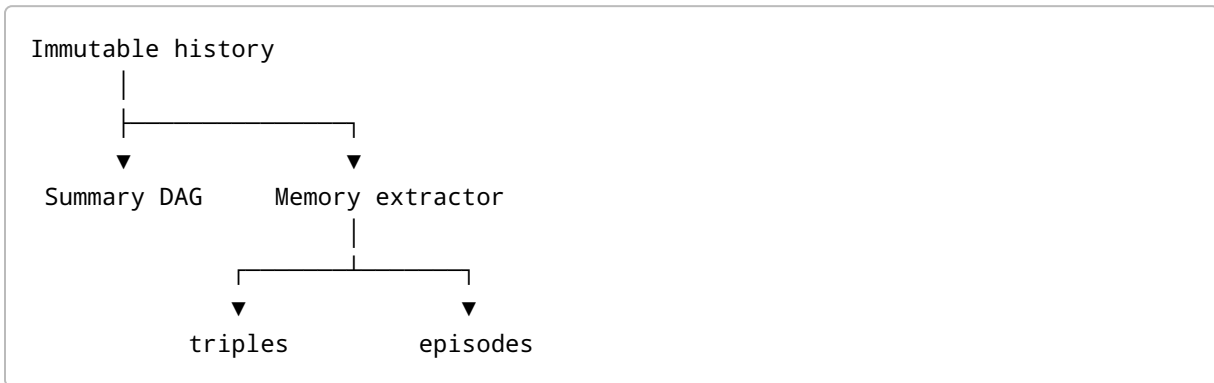
Excellent pairing.

```
raw block
  |
  ▼
ACON compressor
  |
  ▼
optimized summary
  |
  ▼
LCM Summary Node
  |
  └─ pointers → raw block
```

30. MEMORI + LCM combination

LCM preserves historical **events**.

MEMORI extracts reusable **knowledge**.



Do not merge these databases conceptually.

One is:

historical truth

The other is:

derived knowledge

31. Database recommendation

For an offline-first ConAI implementation, begin with:

SQLite

Tables:

sessions
events
focus_sessions
summary_nodes

```
summary_edges
working_memory
episodes
semantic_facts
memory_sources
compression_records
tool_memory
```

Then add:

```
FTS5
```

for lexical retrieval.

For embeddings initially use:

```
FAISS
```

or a local vector index.

LCM itself requires the storage backend to provide transactional writes, referential integrity and indexed search; its reference implementation uses embedded PostgreSQL but explicitly treats the backend as replaceable.

SQLite is therefore reasonable for an early ConAI prototype, although it is my engineering recommendation rather than the LCM paper's reference choice.

32. Suggested project structure

```
app/
|
|— context/
|   |— manager.py
|   |— assembler.py
|   |— budget.py
|   |— policy.py
|   |— pressure.py
|
|— memory/
|   |— event_store.py
|   |— working.py
```

```
| | episodic.py
| | semantic.py
| | tool_memory.py
| | provenance.py
|
| — compression/
| | compressor.py
| | history.py
| | observation.py
| | focus.py
| | optimizer.py
|
| — retrieval/
| | bm25.py
| | vector.py
| | hybrid.py
| | reranker.py
|
| — hierarchy/
| | summary_node.py
| | dag.py
| | compactor.py
| | expansion.py
|
| — agent/
| | planner.py
| | executor.py
| | isolated_reasoner.py
|
| — storage/
| | database.py
| | schema.sql
```

33. Implementation order

Do **not** attempt everything simultaneously.

Phase 1 — Instrumentation

Implement:

```
immutable event logging
token counting
context statistics
latency measurement
```

No intelligent compression yet.

Phase 2 — Deterministic context controller

Implement:

```
soft threshold
hard threshold
recent-context preservation
```

This gives a stable baseline.

Phase 3 — LCM-lite

Implement:

```
immutable store
summary nodes
source pointers
expand()
search()
```

This gives lossless history.

Phase 4 — HyMem

Split:

```
planner
executor
isolated reasoner
```

This should probably come **before sophisticated compression**, because preventing noise is preferable to compressing noise later.

Phase 5 — Structured memory

Implement:

```
working memory
episode memory
tool memory
```

Phase 6 — Focus

Implement:

```
start_focus
complete_focus
focus summary
phase archival
```

Phase 7 — MEMORI-style long-term memory

Implement:

```
fact extraction
semantic triples
conversation/episode summaries
timestamps
hybrid retrieval
provenance
```

Phase 8 — ACON compression

Implement:

history compressor
observation compressor
structured guidelines

Phase 9 — ACON optimization

After collecting benchmark data:

compressed/uncompressed A/B runs
failure analysis
textual gradients
candidate guidelines
UT
optional CO

Phase 10 — Small compressor

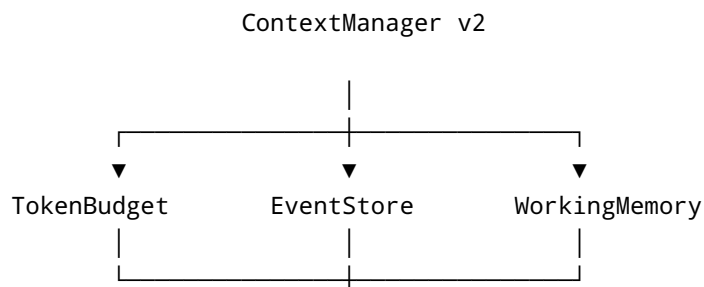
Only after the compressor is demonstrably useful:

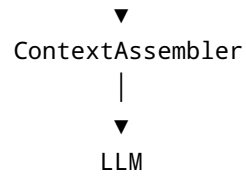
collect teacher outputs
LoRA fine-tune small compressor
benchmark

Do not fine-tune prematurely.

34. What should be implemented immediately

Given the present ConAI architecture, the next context-management milestone should be:





Then introduce:

SummaryDAG

and only afterward:

Planner/Executor isolation

This keeps development incremental and benchmarkable.

35. Benchmarking requirements

Every architecture change should record:

```
task success
total input tokens
total output tokens
peak context
average context
TTFT
generation latency
compression calls
compression latency
compression ratio
memory retrieval latency
number of retrieved memories
tool calls
repeated tool calls
re-exploration events
```

The last metric is especially important.

Focus demonstrates that compression can look good by token ratio while actually causing the agent to redo work.

36. Required ablation tests

Eventually run:

```
A Baseline
B + deterministic trimming
C + LCM
D + HyMem
E + Focus
F + MEMORI
G + ACON
H Full architecture
```

Then additional ablations:

```
without semantic memory
without episodic memory
without tool memory
without observation compression
without history compression
without isolated reasoning
without hybrid retrieval
without summary DAG
```

Otherwise you cannot scientifically claim which subsystem causes an improvement.

37. Key metric: information retention

Compression ratio alone is insufficient.

Create benchmark facts such as:

```
user gives variable X at turn 5
agent needs X at turn 80
```

Measure:

$$Retention = \frac{\text{critical facts correctly recovered}}{\text{critical facts required}}$$

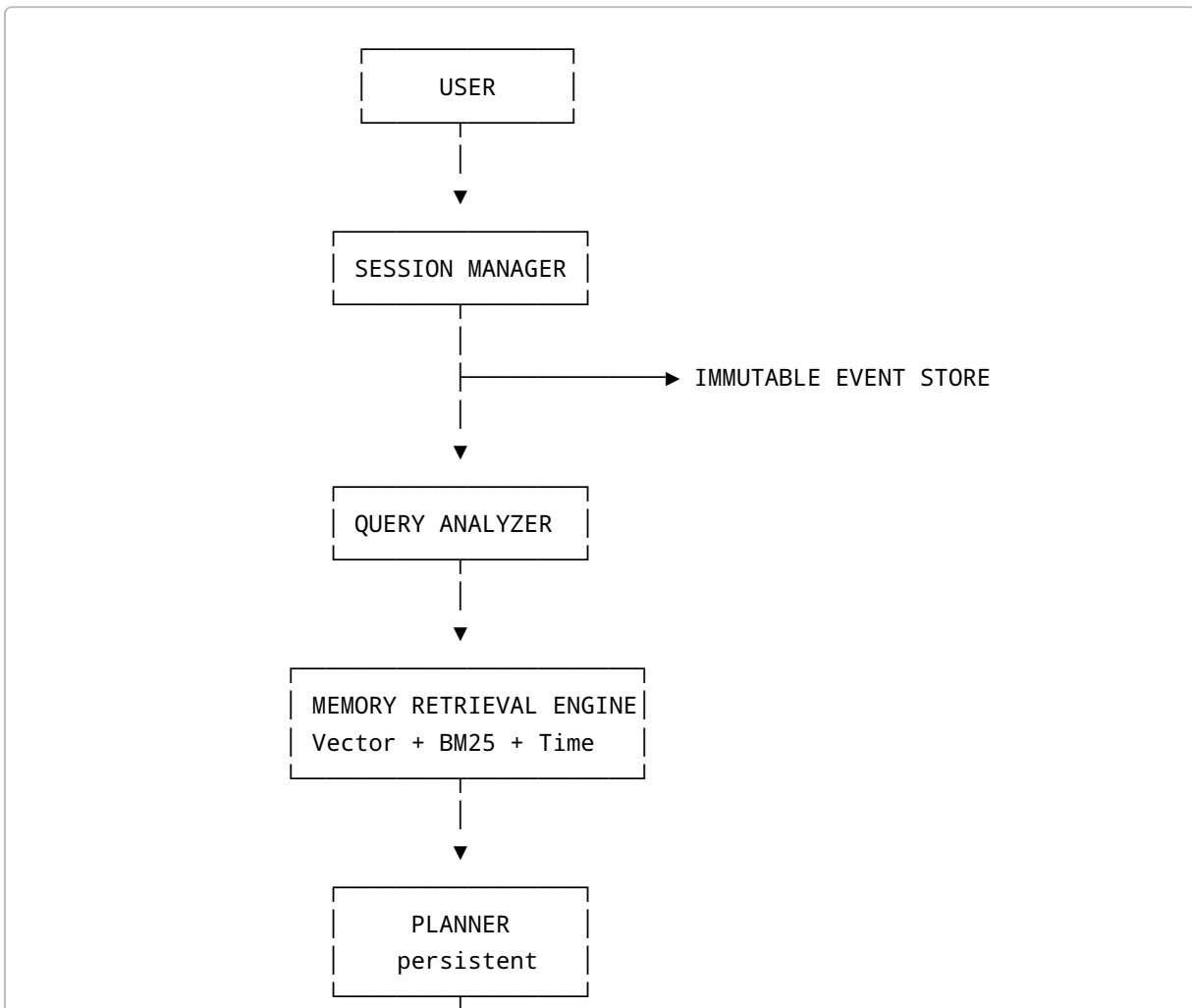
Also evaluate:

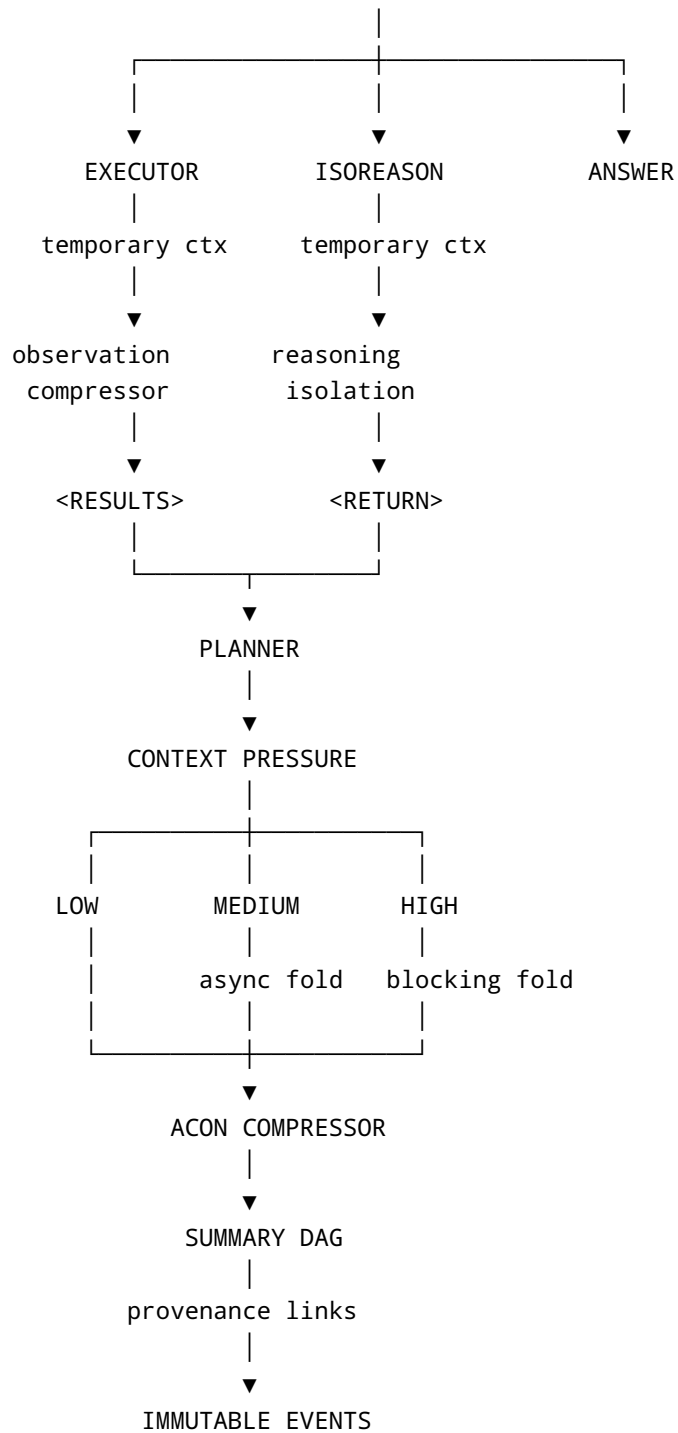
exact-value retention
temporal retention
decision retention
causal retention
artifact/path retention
constraint retention

This directly targets ACON's concern about compression deleting seemingly small but operationally critical information.

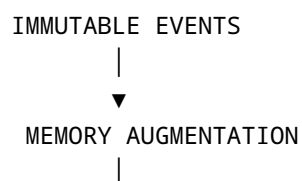
38. Proposed ConAI Context Management Architecture

The final architecture I recommend is:





Meanwhile asynchronously:





39. Where the research opportunity actually is

Simply implementing these five papers would make ConAI an integration project.

The research opportunity lies in the **controller connecting them**.

None of these papers fully solves:

When should we isolate?
When should we retrieve?
When should we compress?
What should we compress?
At what compression ratio?
When should raw context remain?
When should information become long-term memory?
When should a summary be expanded?
Which memory representation should be injected?

That opens a much more interesting ConAI direction:

Adaptive Context Orchestrator

Inputs:

task type
token pressure
context entropy
retrieval confidence
tool activity
failure rate
phase
memory relevance

model capability
latency budget

Outputs:

KEEP
COMPRESS_HISTORY
COMPRESS_OBSERVATION
FOLD
START_FOCUS
END_FOCUS
RETRIEVE
EXPAND
ISOLATE
ARCHIVE

Instead of context management being one algorithm, it becomes a **control problem**.

40. Potential ConAI novelty

A particularly strong architecture would combine:

HyMem
Information Isolation

+

LCM
Lossless Hierarchical History

+

MEMORI
Structured Persistent Memory

+

ACON
Learned Compression Policy

+

Focus

Semantic Phase Consolidation

+

ConAI
Adaptive Context Controller

The potential novelty is therefore not:

"We summarize long context."

That is already saturated.

A stronger claim would be:

A multi-timescale adaptive context-management architecture that dynamically selects information isolation, lossless hierarchical compaction, structured persistent-memory retrieval, and task-conditioned compression according to context pressure and task state.

That is substantially more defensible as a research direction.

41. Most important lessons from all five papers

Lesson 1

Context and memory are not the same thing.

Context is what the LLM sees now.

Memory is what the system can recover later.

Lesson 2

Never make compression destructive.

LCM provides the right principle:

compress representation
preserve source

Lesson 3

Prevent pollution before compressing it.

HyMem's planner/executor isolation should sit upstream of compression.

Lesson 4

Compression must preserve operational state.

ACON shows why generic summarization is insufficient.

Lesson 5

The agent should understand semantic task boundaries.

Focus provides a useful mechanism for phase-level consolidation.

Lesson 6

Persistent memory should be structured before retrieval.

MEMORI demonstrates the value of triples plus summaries rather than raw-conversation RAG.

Lesson 7

Different information needs different lifetimes.

tool output	→ seconds
working state	→ minutes
focus summary	→ task
episode	→ sessions
semantic fact	→ months
raw history	→ permanent/archive

Lesson 8

Compression itself costs tokens.

Focus explicitly observes this "cognitive tax"; compression only makes sense when future savings exceed its cost.

Lesson 9

Small models can handle compression.

ACON demonstrates that the expensive reasoning model need not also be the compressor.

That is particularly useful for a local-first ConAI architecture.

Lesson 10

The final system should be adaptive, not threshold-only.

Thresholds are excellent safety rails.

They should not be the complete intelligence of the memory system.

42. Final implementation recommendation

For ConAI, I would build the research architecture in this exact dependency order:

```
1. Immutable Event Store
  ↓
2. Token/Context Telemetry
  ↓
3. Context Assembler
  ↓
4. Soft/Hard Context Controller
  ↓
5. Lossless Summary DAG
  ↓
6. Planner/Executor Isolation
  ↓
7. Working/Episode/Tool Memory
  ↓
8. Focus Sessions
  ↓
9. Structured Semantic Memory
  ↓
```

10. Hybrid Retrieval
- ↓
11. History + Observation Compressor
- ↓
12. Failure-driven ACON Optimizer
- ↓
13. Adaptive Context Orchestrator
- ↓
14. Small-model Compressor Distillation

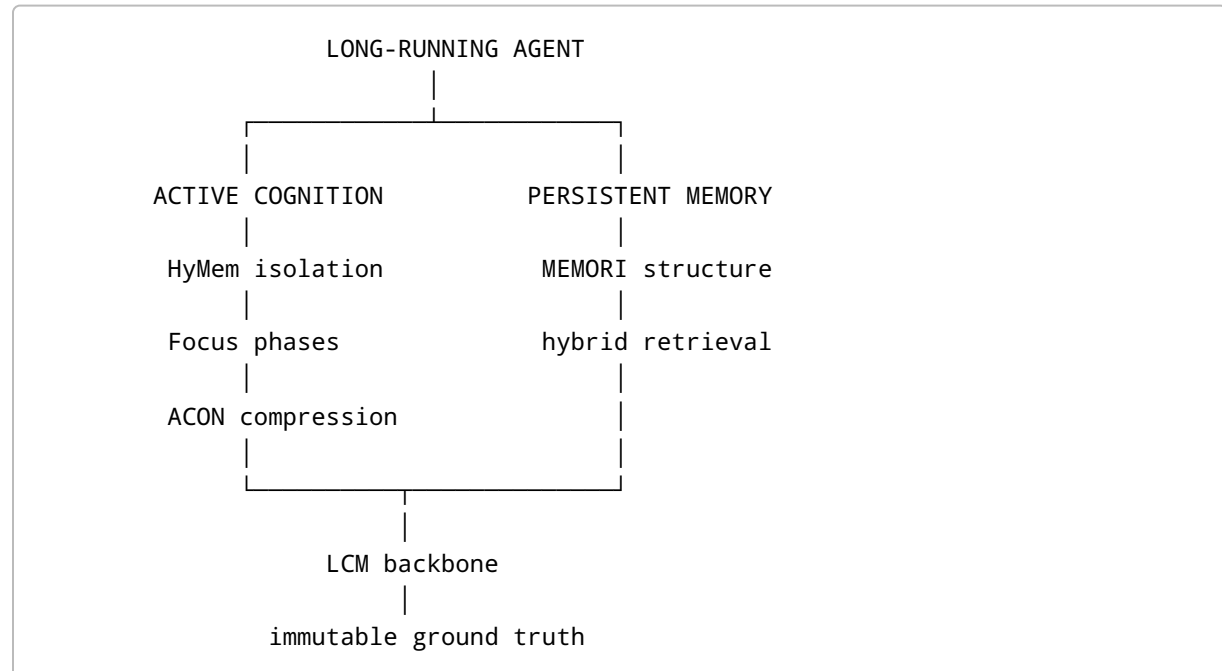
This order matters.

If semantic memory, ACON, Focus, HyMem and hierarchical compression are all implemented simultaneously, failures become almost impossible to attribute.

With staged implementation, every stage can be benchmarked against the previous architecture.

43. Bottom line

The papers collectively point toward a clear conclusion:



HyMem should define how information flows.

LCM should define how historical information is preserved.

MEMORI should define how reusable knowledge is represented.

Focus should define semantic task-phase boundaries.

ACON should define how compressed representations improve from failures.

And the ConAI-specific research layer should sit above all five:

an adaptive controller deciding which memory/context operation to execute, based on the current task, information type, context pressure, retrieval confidence, and observed failures.

That gives ConAI a coherent architecture rather than a collection of memory features.